

# JPMC Java + Spring Boot

Top 50 Interview Questions & Answers — Student CRUD Based

Preparation note: this document uses the supplied Student CRUD notes as the core syllabus and adds JPMC-oriented Java/Spring/REST/JPA interview questions. Public interview reports show that JPMC Java roles have included Core Java, Spring Boot, Hibernate/JPA, SQL, REST/web services, code review and scenario-based project questions. The exact questions vary by role, interviewer and experience level.

## Table of Contents

1. Student CRUD & REST Fundamentals — Q1–Q15
2. Spring, Dependency Injection & JPA — Q16–Q27
3. CRUD, Soft Delete & API Design — Q28–Q39
4. Production & Code-Review Questions — Q40–Q50
5. Rapid Revision
6. JPMC Preparation Notes

## 1. Student CRUD & REST Fundamentals

### Q1. What is CRUD?

CRUD stands for Create, Read, Update and Delete. In this project, POST creates a Student, GET reads Student data, PUT updates it, and DELETE permanently removes it. PATCH is additionally used for soft delete.

### Q2. Explain the architecture of your Student CRUD application.

The flow is Client/Postman → Controller → Service → Repository → MySQL. The Entity represents the Student data. Controller handles HTTP, Service handles application logic, Repository handles persistence.

### Q3. Why do we use Controller, Service and Repository layers separately?

Separation of responsibilities. Controller handles HTTP, Service contains application logic, and Repository handles database access. This reduces coupling and makes the code easier to test and maintain.

### Q4. What does @RestController do?

@RestController marks a class as a REST controller. Its methods handle HTTP requests and their return values can be written directly to the response body.

### Q5. What is the purpose of @RequestMapping("/api/student")?

It defines the common base URL for all endpoints in the controller. For example, @GetMapping("/{id}") becomes GET /api/student/{id}.

### Q6. What is the difference between @GetMapping, @PostMapping, @PutMapping, @PatchMapping and @DeleteMapping?

@GetMapping reads data, @PostMapping creates, @PutMapping replaces/updates a resource, @PatchMapping performs a partial/state change, and @DeleteMapping deletes a resource.

### Q7. What does @RequestBody do?

It tells Spring to read the HTTP request body and convert JSON into the Java object used by the controller method. Jackson performs the JSON-to-object conversion in a typical Spring Boot REST application.

### Q8. What does @PathVariable do?

It extracts a value from the URL path. In GET /api/student/5, @PathVariable("id") Long id gives the method the value 5.

### Q9. Why use ResponseEntity?

ResponseEntity lets the controller control the HTTP response body, status code and headers. The supplied project uses it for 201 CREATED, 200 OK and 404 NOT\_FOUND responses.

### Q10. Why does the create endpoint return 201 CREATED?

201 CREATED is the HTTP status used by the supplied code for successful resource creation. It communicates that a new Student resource was created.

### Q11. Explain the POST request end-to-end.

JSON reaches the controller → @RequestBody creates Student → controller calls Service.addStudent() → service sets isDeleted=false → repository.save() persists it → saved Student returns through the layers → controller returns 201 CREATED.

### Q12. What is @Entity?

@Entity tells JPA that Student is a persistent entity. Hibernate can map the entity fields to database columns.

### Q13. What is @Id?

@Id marks the primary-key field. In Student, id uniquely identifies a record.

### Q14. What does GenerationType.IDENTITY mean?

It uses the database identity/auto-increment mechanism to generate the primary-key value. The application normally does not need to manually generate the ID for a new record.

### Q15. What is ORM?

Object-Relational Mapping maps Java objects to relational database data. Here, Student is mapped to a database table and its fields map to columns.

## 2. Spring, Dependency Injection & JPA

### Q16. What is JPA and what is Hibernate?

JPA is the Java persistence specification/API. Hibernate is a commonly used JPA implementation. In this project, Spring Data JPA works with Hibernate to persist Student entities.

### Q17. What is JpaRepository?

JpaRepository is a Spring Data interface that provides common persistence operations such as save(), findById(), existsById() and deleteById(), along with support for derived query methods.

### Q18. Why does StudentRepository extend JpaRepository<Student, Long>?

Student is the entity type and Long is the type of its primary key. This gives the repository CRUD operations for Student records with Long IDs.

### Q19. What is a derived query method?

It is a repository method whose name describes the query. Spring Data parses the method name and creates the query. Example: findByIdAndIsDeletedIsFalse(Long id).

### Q20. Explain findByIdAndIsDeletedIsFalse().

It asks the repository to find a Student with the supplied ID and with isDeleted=false. Therefore a soft-deleted record is excluded from this lookup.

### Q21. What is Optional<Student> and why use it?

Optional represents that a query may return a Student or no Student. The service can use isPresent() or isEmpty() before calling get().

### Q22. What is dependency injection in this project?

A class receives a dependency from Spring instead of creating it with new. Controller receives Service and Service receives StudentRepository through constructors.

---

### Q23. Why is constructor injection preferred here?

The dependency is supplied when the object is constructed, making the dependency explicit. The Service also stores its repository reference in a final field.

### Q24. What is the difference between IoC and DI?

IoC is the broader idea that Spring takes control of object creation and management. DI is one technique through which Spring supplies required dependencies to objects.

### Q25. What does @Service do?

@Service marks the class as a Spring-managed service component. Spring can create it and inject it into the Controller.

### Q26. What does @Repository do?

@Repository marks a persistence component. In this project, StudentRepository provides the database-access abstraction.

### Q27. Explain the GET-by-ID flow.

GET /api/student/{id} reaches the controller → controller calls findStudentById() → repository searches for the ID with isDeleted=false → service returns Student or null → controller returns 200 OK or 404 NOT\_FOUND.

## 3. CRUD, Soft Delete & API Design

### Q28. Why return 404 NOT\_FOUND when a Student does not exist?

404 indicates that the requested resource could not be found. The supplied controller uses this status when the service returns null or the Student is deleted.

### Q29. How does GET-all work in this project?

GET /api/student/all calls the service. The service obtains non-deleted records from the repository, builds a List<Student>, and returns the list to the controller for JSON serialization.

### Q30. What is soft delete?

Soft delete means the row remains in the database but is marked as deleted. This project uses isDeleted=true. Normal read queries can exclude those records.

### Q31. Why would you use soft delete instead of physical delete?

Soft delete preserves the row and its history while allowing the application to hide it from normal reads. This can be useful when records may need auditing or recovery.

### Q32. Explain the soft-delete flow in this code.

PATCH /api/student/soft-delete/{id} → service finds an active Student → setDeleted(true) → save/update the record → return success. The row remains in the database.

### Q33. What is permanent delete in this project?

The DELETE endpoint calls existsById(id) and then deleteById(id). This removes the database row rather than changing isDeleted.

### Q34. What is the difference between PATCH and DELETE in this project?

PATCH is used for the soft-delete state change: isDeleted becomes true. DELETE is used for permanent removal of the row.

### Q35. Explain the PUT update flow.

PUT /api/student/update/{id} receives the ID and JSON body → service finds the existing active Student → copies the supplied fields → repository.save() persists the changes → controller returns an update message.

---

### Q36. What issue do you see in the update method?

The supplied code calls updated\_Student.setId(s.getId()). When updating by path-variable ID, it is normally safer to preserve the existing entity ID and update only editable fields. A different body ID can create an identifier mismatch.

**Q37. What issue do you see in the getAll repository method?**

The supplied method is `Student[] findByAndIsDeletedFalse()`. The naming is unusual for a Spring Data derived query. A conventional form would normally be `findByIsDeletedFalse()`. It should be verified by compiling/testing the project.

**Q38. What is Jackson's role in a REST API?**

Jackson commonly converts JSON request bodies into Java objects and Java return objects into JSON responses. That is why a Student object can be accepted from and returned to Postman as JSON.

**Q39. How does Spring convert a returned Student into JSON?**

The REST response is handled by Spring's HTTP message conversion infrastructure, commonly using Jackson. The Student object's fields are serialized into JSON.

## 4. Production & Code-Review Questions

**Q40. How would you implement centralized exception handling?**

A common Spring approach is `@ControllerAdvice` with `@ExceptionHandler` methods. This keeps exception-to-HTTP-response mapping in one place rather than duplicating it across controllers.

**Q41. How would you add pagination to this Student API?**

Use Spring Data pagination, for example a repository method accepting `Pageable` and returning `Page<Student>`. The controller can accept page and size parameters and return the requested page.

**Q42. How would you secure this REST API?**

Use authentication and authorization, HTTPS, input validation, controlled access to endpoints, safe credential storage, and appropriate Spring Security configuration. Do not hard-code secrets in source control.

**Q43. Where should the database password be stored in production?**

It should normally be externalized using environment variables, a secrets manager, deployment configuration, or another secure configuration mechanism rather than committing it directly to `application.properties`.

**Q44. What is @Transactional and where can it help?**

`@Transactional` defines a transaction boundary. A group of database operations can succeed together or roll back together on failure. It is typically placed around service operations that must be atomic.

**Q45. What happens if repository.save() fails?**

The persistence operation throws an exception. In production code, the exception should be handled through appropriate transaction boundaries and centralized exception handling rather than returning misleading success messages.

**Q46. How would you validate Student input?**

Use Bean Validation annotations such as `@NotBlank`, `@Email`, `@Min` and `@Max` on the entity/request DTO and use `@Valid` in the controller. Invalid requests can then return a validation error response.

**Q47. Why might you use DTOs instead of exposing Student directly?**

DTOs separate the API contract from the persistence entity. They help prevent exposing internal fields, allow request/response-specific validation, and reduce accidental changes to database models.

**Q48. What code-quality issue is visible in the controller's update method?**

The variable `boolean f=false` is unused. It should be removed. Also, returning 200 OK for a missing record is less expressive than returning an appropriate error status.

**Q49. How would you improve the soft-delete implementation?**

The current code changes `isDeleted` and calls `updateData()`, which also copies multiple Student fields. A simpler implementation would load the entity, set `isDeleted=true`, call `repository.save(entity)`, and return the result.

**Q50. Explain this project as if the interviewer asks you to walk through it.**

Student is the JPA entity. Postman sends HTTP requests to the REST Controller. The Controller maps the request and delegates to Service. Service contains the application logic and calls StudentRepository. JpaRepository uses JPA/Hibernate to interact with MySQL. Reads exclude soft-deleted records, PATCH performs soft delete, and DELETE performs permanent deletion.

## 5. Rapid Revision

| Topic             | One-line answer                          |
|-------------------|------------------------------------------|
| CRUD              | Create, Read, Update, Delete.            |
| Controller        | HTTP request/response layer.             |
| Service           | Application/business logic.              |
| Repository        | Persistence/database access.             |
| @Entity           | JPA persistent entity.                   |
| @RequestBody      | JSON request body → Java object.         |
| @PathVariable     | URL path value → Java parameter.         |
| JpaRepository     | Spring Data repository CRUD abstraction. |
| Optional          | Value may be present or absent.          |
| Soft delete       | Mark deleted; keep row.                  |
| DELETE            | Permanently remove row in this project.  |
| ResponseEntity    | Control response status/body.            |
| @Transactional    | Defines transaction boundary.            |
| @ControllerAdvice | Centralized exception handling.          |

## 6. JPMC Preparation Notes

Public interview reports are not a guaranteed question bank. They indicate recurring areas reported by candidates rather than an official JPMC syllabus.

For Java/Spring roles, reported topics include Core Java, Java 8, Spring Boot, Hibernate/JPA, SQL, REST/web services, collections, concurrency and project/scenario discussion.

Recent public candidate reports also describe code-review exercises involving Java/Spring code, exception handling, annotations, SRP and code quality. These reports support practicing the CRUD code in this document as a review exercise.

A useful interview habit is to explain every line in the CRUD flow: why the annotation exists, where the request goes next, what the repository query does, what happens when data is missing, and which HTTP status is returned.

Source note: JPMC-specific preparation points were cross-checked against public interview reports and guides. They should be treated as reported experiences, not as guaranteed questions.